

# Assignment: Automation Rules Engine (Track 2: Advanced)

Claude Code Training — Track 2 April 24, 2026

## Overview

Track 2 assumes you have already completed the beginner track and are comfortable with the CLAUDE.md / design / plan / architecture / TDD loop. It raises the bar in two specific directions. First, orchestration: you are expected to use at least two distinct subagents, and to run at least one explicit review loop in which one subagent's output is critiqued by another subagent before it is integrated. The point is not to invoke subagents ceremonially — it is to demonstrate that you know when a second pair of eyes catches things the first pass misses, and to leave audit evidence of that loop in your repo. Second, rigor: you will use either the superpowers or the cavern plugin to shape your architecture document, and your TDD evidence must include a coverage report and, ideally, a mutation-testing pass for bonus credit.

This is not a bigger version of the beginner track. The grading rubric weights orchestration, plugin evidence, and TDD discipline more heavily than raw feature completeness. A student who ships a partial feature with clean subagent evidence, a tight architecture, and a full coverage report will score higher than a student who ships every bonus feature but skips the review loops. Budget your time accordingly. If you find yourself cutting corners on the plan-review or architecture-review step to gain more implementation time, stop and reallocate — the graders are calibrated to notice.

## Time Budget

Total: 5-6 hours.

| Phase                                  | Time    |
|----------------------------------------|---------|
| Setup + starter repo                   | 20 min  |
| CLAUDE.md files (manual + /init)       | 30 min  |
| Design phase                           | 45 min  |
| Plan phase (subagent)                  | 45 min  |
| Architecture phase (subagent + review) | 60 min  |
| TDD implementation                     | 150 min |
| HTML workflow report                   | 30 min  |

## What You Will Build

### Rule definition

- A Rule has: `id`, `name`, `event` (enum: `task_created` / `task_status_changed` / `task_labeled`), `conditions` (JSON tree of ANDs/ORs of predicate leaves), `actions` (ordered list of action specs), `is_active`, `created_by`, `timestamps`.
- `event` selects when the rule can fire. A rule only evaluates against events whose type matches.
- `conditions` is a boolean tree. Leaves have the shape `{ predicate: "status_equals" | "label_equals" | "title_contains" | "assigned_to_equals", value: any }`. Composites have the shape `{ op: "AND" | "OR", children: [tree, ...] }`. Trees may nest arbitrarily; enforce a max depth in validation.
- `actions` is an ordered list. Types:
  - `{ type: "set_status", status: "...", }`
  - `{ type: "add_label", label: "...", }`
  - `{ type: "notify_webhook", url: "https://...", secret?: "...", }`
  - `{ type: "send_in_app_notification", to: "assignee|creator|user_id", body: "...", }`

## Events

- Any mutation to a Task that changes status, assignment, or labels emits an internal event onto a queue.
- Events include both the pre-mutation and post-mutation task snapshots. Rules that need to distinguish "moved to done" from "was already done" rely on the delta.
- Rules whose event matches the event type are selected as candidates and dispatched to a background worker for evaluation.

## Rule matcher

- Given a rule and an event, the matcher returns true or false.
- The matcher must handle deeply nested condition trees. A depth guard should reject trees deeper than the documented limit before evaluation, not after.
- The matcher is a pure function of `(rule, event)`. No I/O, no clock reads, no database calls. TDD required.

## Action executor

- On match, actions run in order.
- `notify_webhook` posts a JSON payload to the given URL. It must retry with exponential backoff up to 3 total attempts on non-2xx responses. It must record every delivery attempt (URL, status code, attempt number, latency, error).
- Actions that fail must not roll back earlier actions in the same run. Idempotency across retries is the rule author's problem, not the engine's.
- TDD required, including the retry ladder. Assert both the number of attempts and the interval schedule.

## Rule builder UI

- List page: table of rules with an active toggle, name, event, and last-fired timestamp.
- Editor: name field, event dropdown, condition tree builder (nested groups with add-child buttons for both new leaves and new groups), ordered actions list.
- Dry-run: pick a Task, evaluate the rule against a synthetic event derived from that Task, and show which condition leaves matched and which actions would have fired — without persisting anything.

## Bonus features

Each is worth 5 points, up to +15.

- Rule execution history log at `/rules/{id}/history` — every fire, with matched conditions and per-action outcomes.
- Global admin view of webhook delivery attempts (URL, status code, latency, retry count) at `/admin/webhooks`.
- Dry-run supports arbitrary hand-typed event JSON, not just events derived from an existing Task.

## Choose Your Starter Repo

| Stack   | Path                                               | Test / worker command                                              |
|---------|----------------------------------------------------|--------------------------------------------------------------------|
| Laravel | <code>starter-repos/laravel-starter/</code>        | <code>php artisan test + php artisan queue:work</code>             |
| FastAPI | <code>starter-repos/python-fastapi-starter/</code> | <code>pytest + arq</code> or your background runner of choice      |
| Next.js | <code>starter-repos/nextjs-starter/</code>         | <code>npm test + queue via bullmq / trigger.dev / your call</code> |

You choose the background runner. Justify the choice in your architecture document — the graders want to see that you weighed at least one alternative.

## Workflow

### Step 1 — CLAUDE.md files

The beginner track asked for CLAUDE.md files that captured stack, layout, and commands. Track 2 raises the bar. Your project CLAUDE.md must explicitly capture:

- The rule tree shape and where it is validated (name the class or module).
- Which pieces are pure functions (and must therefore be TDD'd to high coverage) versus side-effectful (and therefore need integration tests instead of mocking).
- How to run the worker locally, including which queue name to consume.

Submit both the manual version and the `/init` version, and write a short reflection in `docs/claude-md-notes.md` on where they diverged and why.

### Step 2 — Design phase (45 min)

Deliverable: `design/design-brief.md` with:

- Wireframes for the rule list, the rule editor with the nested condition builder, and the dry-run panel.
- A component tree.
- Two error states you designed for explicitly — one for an invalid condition tree (e.g., missing predicate, unknown op, too deep) and one for a runtime action failure surfaced back to the user.

### Step 3 — Plan phase (subagent)

Invoke a planning subagent (e.g., the Plan agent) to draft the implementation plan. Then invoke a second subagent — a general-purpose agent in a "critique this plan" mode — to review the plan and propose changes. Deliverable: `docs/plan.md` (the final, revised plan) and `docs/plan-review.md` (the critique).

Both must be committed. If you disagree with the reviewer, note the disagreement in `plan.md` rather than silently dropping the feedback.

### Step 4 — Architecture phase (subagent + plugin)

Produce the architecture document with either the `superpowers` plugin or the `cavern` plugin — your choice, but state it. The plugin's output should visibly shape the doc; if a grader can't tell which sections came from the plugin, the plugin evidence score suffers.

Then invoke a subagent to review the architecture document against `docs/plan.md` for consistency (concretely: "does the architecture cover every step the plan lists, and does it introduce anything the plan does not?"). Deliverables: `docs/architecture.md` and `docs/architecture-review.md`.

### Step 5 — Build TDD-style

Three TDD targets. Each requires a failing-test commit before the corresponding implementation commit — the graders will look for this in the git log.

- Rule matcher against nested condition trees. At least 10 cases, including leaf-only, nested AND, nested OR, mixed nesting, depth-guard rejection, invalid predicate, invalid op, empty children, event-type mismatch, and one case with 6+ levels of nesting.
- Action executor. Cover order-of-execution, one-action-fails-others-still-run, and per-action outcome recording.
- Webhook delivery with retry. Mock the HTTP client; assert the exponential backoff schedule (both attempt count and intervals).

Coverage recommendation: aim for 85%+ on the rule matcher and action executor modules. Attach a coverage report — HTML or JSON — to the workflow report.

### Step 6 — HTML workflow report (30 min)

Same six sections as the beginner track, plus:

- Section 7: "Subagent Orchestration Diagram" — a short flow showing which subagent produced which artifact and which review loop consumed it.
- Section 8: "Plugin Usage Evidence" — screenshot or excerpt showing which plugin command you invoked and how its output shaped the architecture doc.

## Deliverables Checklist

- User `CLAUDE.md` — manual + `/init` versions
- Project `CLAUDE.md` — manual + `/init` versions
- `docs/claude-md-notes.md` (reflection)
- `design/design-brief.md`
- `docs/plan.md`
- `docs/plan-review.md`
- `docs/architecture.md`
- `docs/architecture-review.md`
- Rule matcher, action executor, and webhook retry all with a failing-test commit before the implementation commit
- Coverage report attached under `report/coverage/` (HTML or JSON)

- `report/workflow-report.html` — 8 sections including the orchestration diagram and plugin evidence
- Bonus items listed and completed (if any)

## Grading Criteria (short version)

Full breakdown in `RUBRIC.md`. Total 100 points.

- CLAUDE.md quality: 10
- Design brief: 5
- Plan doc + review: 10
- Architecture doc + review: 15
- Feature correctness: 20
- TDD evidence + coverage: 15
- Subagent orchestration quality: 10
- Plugin usage evidence: 5
- HTML workflow report: 10

Bonus: up to +15 for the extra features listed above.